SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology
Department of Agricultural Engineering

AGE 219: Basics of Computer Programming

Lecture Compendium 9: Data Analysis and Visualization — From Raw Sensors to Engineering Intelligence

---

Prepared by:

Dr. Kadeghe Fue, PhD, P.Eng (T)

Target Audience: Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
Primary Tools: Python 3.x, Pandas, and Matplotlib
Location: Morogoro, Tanzania

Data is often described as the 'new oil.' However, raw data, like crude oil, is useless until it is refined. This lecture teaches you how to use Pandas to clean and analyze massive agricultural datasets and Matplotlib to visualize the results. These are the tools that allow an engineer to prove hypotheses, optimize machinery, and communicate findings to stakeholders.

Contents

1 Introduction: The Informatics Revolution in Agriculture **2**
1.1 The Analytic Workflow . . . 2

2 Introduction to Pandas: The Dataframe Architecture **2**
2.1 Importing the Library . . . 2
2.2 Creating a DataFrame . . . 2

3 Data Hygiene: Cleaning and Pre-processing **3**
3.1 Handling Missing Data (`NaN`) . . . 3
3.2 Filtering for Outliers . . . 3

4 Data Transformation and Aggregation **3**
4.1 Calculating Derived Metrics . . . 3
4.2 The `groupby` Operation . . . 3

5 Matplotlib: The Anatomy of an Engineering Plot **3**
5.1 Importing Matplotlib . . . 3
5.2 Basic Plot Elements . . . 4

6 Common Agricultural Visualizations **4**
6.1 1. Line Plots: Time-Series Trends . . . 4
6.2 2. Scatter Plots: Relationship Analysis . . . 4

7 Case Study: Analyzing Machinery Telematics Data **4**

8 Styling for Publication and Reports **5**

9 Best Practices for Agri-Engineering Analytics **5**

10 Summary **6**

11 Glossary of Terms **7**

12 Extended Self-Assessment Exercises **8**
12.1 Exercise 1: Irrigation Data Analysis . . . 8
12.2 Exercise 2: Weather Data Cleaning . . . 8
12.3 Exercise 3: The Growth Curve Plot . . . 8
12.4 Exercise 4: Machinery Efficiency Heatmap (Advanced) . . . 8

1 Introduction: The Informatics Revolution in Agriculture

As an agricultural engineer in the 21st century, you are no longer just managing physical hardware; you are managing a flood of information. A single harvest season can produce millions of data points from GPS-enabled combine harvesters, weather stations, and satellite imagery.

Traditional spreadsheet software (like Microsoft Excel) is excellent for small datasets, but it struggles with "Big Data." When you have 500,000 rows of tractor telematics data, Excel becomes slow and difficult to automate. This is where **Pandas** and **Matplotlib** become indispensable.

1.1 The Analytic Workflow

Professional engineering data analysis follows a strict pipeline:

1. **Ingestion:** Loading data from CSV, SQL, or IoT APIs.
2. **Cleaning:** Removing errors, handling missing values (`NaN`), and fixing units.
3. **Transformation:** Calculating new metrics (e.g., converting Speed and Width to Field Capacity).
4. **Aggregation:** Summarizing data (e.g., Average yield per crop variety).
5. **Visualization:** Creating charts that reveal patterns and anomalies.

2 Introduction to Pandas: The Dataframe Architecture

Pandas is built on top of `NumPy`. While NumPy handles mathematical arrays, Pandas provides the **DataFrame**—a two-dimensional, size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns).

2.1 Importing the Library

By convention, we always import pandas as `pd`.

```
import pandas as pd
import numpy as np
```

2.2 Creating a DataFrame

In agricultural applications, DataFrames usually come from CSV files, but you can create them from Dictionaries.

```
# Modeling a small farm inventory
data = {
    'Crop': ['Maize', 'Maize', 'Rice', 'Rice', 'Beans'],
    'Plot_ID': [101, 102, 201, 202, 301],
    'Yield_t_ha': [4.5, 4.2, 5.8, 6.1, 1.8],
    'Irrigated': [True, False, True, True, False]
}

df = pd.DataFrame(data)
print(df.head()) # Displays the first 5 rows
```

3 Data Hygiene: Cleaning and Pre-processing

Sensor data is rarely perfect. Wireless interference might cause missing readings, or a faulty sensor might report a temperature of 500°C.

3.1 Handling Missing Data (`NaN`)

In Pandas, missing values are represented as `NaN` (Not a Number).

- `df.isnull().sum()`: Count missing values.
- `df.dropna()`: Remove rows with missing data.
- `df.fillna(value)`: Replace missing data with a constant or the average.

3.2 Filtering for Outliers

Engineers must define "Physical Constraints." If a soil moisture sensor reports a value $> 100\%$, it is physically impossible and must be filtered out.

```
# Filtering for valid data
clean_df = df[df['Yield_t_ha'] > 0]
```

4 Data Transformation and Aggregation

The power of Pandas lies in its ability to perform "Vectorized Operations." You can multiply an entire column by a constant without writing a loop.

4.1 Calculating Derived Metrics

```
# Assume we have Area in Hectares and Yield in tons/ha
df['Total_Harvest_Tons'] = df['Yield_t_ha'] * df['Area_ha']
```

4.2 The `groupby` Operation

This is the most common task in agricultural reporting: "What is the average yield per crop type?"

```
# The 'Split-Apply-Combine' strategy
summary = df.groupby('Crop')['Yield_t_ha'].mean()
print(summary)
```

5 Matplotlib: The Anatomy of an Engineering Plot

Data visualization is not just about making "pretty pictures"; it is about **Communication**. A good engineering graph must be self-explanatory, labeled with units, and high-resolution.

5.1 Importing Matplotlib

```
import matplotlib.pyplot as plt
```

5.2 Basic Plot Elements

A professional plot requires:

- **Title:** Clear description of the experiment.
- **X and Y Labels:** Variable names and units (e.g., "Time [s]").
- **Legend:** If multiple datasets are shown.
- **Grid:** To help estimate values visually.

6 Common Agricultural Visualizations

6.1 1. Line Plots: Time-Series Trends

Used for weather data, grain drying curves, and growth rates.

```
days = [1, 2, 3, 4, 5, 6, 7]
temp = [22, 24, 28, 25, 26, 30, 31]

plt.figure(figsize=(10, 5))
plt.plot(days, temp, marker='o', color='green', linestyle='--')
plt.title('Daily Greenhouse Temperature Trend')
plt.xlabel('Day of Week')
plt.ylabel('Temperature [Celsius]')
plt.grid(True)
plt.show()
```

6.2 2. Scatter Plots: Relationship Analysis

Used to identify correlations, such as "Does fertilizer rate affect yield?"

```
fertilizer_kg = [50, 100, 150, 200, 250]
yield_tons = [3.2, 4.1, 4.8, 5.0, 5.1]

plt.scatter(fertilizer_kg, yield_tons, color='brown')
plt.title('Fertilizer Response Curve')
plt.xlabel('Nitrogen Application [kg/ha]')
plt.ylabel('Yield [tons/ha]')
plt.show()
```

7 Case Study: Analyzing Machinery Telematics Data

Engineering Problem: A farm manager at SUA is reviewing telematics data from a fleet of tractors. We need to identify if there is a relationship between **Engine Load (%)** and **Fuel Consumption (L/hr)**. Specifically, we want to find the most efficient operating range to minimize fuel costs.

```
# --- 1. Data Generation (Simulated Telematics Log) ---
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Creating a realistic dataset
np.random.seed(42)
load = np.random.uniform(20, 100, 100) # 20% to 100% load
# Fuel increases with load but with some variation (noise)
```

```
fuel = (load * 0.4) + np.random.normal(0, 2, 100)

df_telematics = pd.DataFrame({
    'Engine_Load_Pct': load,
    'Fuel_L_hr': fuel
})

# --- 2. Data Analysis ---
# Calculate the 'Specific Fuel Consumption' (Liters per percent load)
df_telematics['L_per_Pct'] = df_telematics['Fuel_L_hr'] / df_telematics['Engine_Load_Pct']

avg_consumption = df_telematics['Fuel_L_hr'].mean()
print(f"Fleet Average Fuel Consumption: {avg_consumption:.2f} L/hr")

# --- 3. Visualization ---
plt.figure(figsize=(12, 6))

# Main Scatter Plot
plt.scatter(df_telematics['Engine_Load_Pct'], df_telematics['Fuel_L_hr'],
            alpha=0.6, label='Individual Data Points')

# Trend Line (Simple Linear Regression logic)
m, b = np.polyfit(df_telematics['Engine_Load_Pct'], df_telematics['Fuel_L_hr'], 1)
plt.plot(df_telematics['Engine_Load_Pct'], m*df_telematics['Engine_Load_Pct'] + b,
         color='red', linewidth=2, label='Linear Trend')

plt.title('Tractor Performance: Engine Load vs. Fuel Rate')
plt.xlabel('Engine Load [%]')
plt.ylabel('Fuel Consumption [L/hr]')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

8 Styling for Publication and Reports

As a professional engineer, you cannot submit default Matplotlib charts. You should use styles and high-quality DPI settings for your final BSc project or company reports.

```
# Using a professional style
plt.style.use('ggplot')

# Setting high resolution for printing
plt.rcParams['figure.dpi'] = 300

# Saving the figure to a file
# plt.savefig('tractor_efficiency_map.png', bbox_inches='tight')
```

9 Best Practices for Agri-Engineering Analytics

1. **Unit Consistency:** Always verify that your units match across datasets (e.g., don't mix m^3 and Liters).
2. **Label Everything:** A graph without labels is a visual lie.
3. **Data Backup:** Never modify your "Source CSV" file. Load it into Pandas, perform changes in memory, and save a *new* CSV if needed.

4. **Vectorization:** Avoid using `for` loops to process Pandas columns. Use `df['A'] * df['B']` instead; it is 100x faster.

10 Summary

In this lecture, we have mastered the tools of the modern ”Data-Driven Engineer.” You have learned:

- How **Pandas DataFrames** provide a structured way to handle large tabular datasets.
- Techniques for **Data Cleaning** and filtering to ensure engineering precision.
- The logic of **Aggregation** using `groupby`.
- The fundamentals of **Matplotlib** to create line and scatter plots.
- Professional formatting standards for engineering reports.

In our final lecture, **Lecture 10**, we will integrate everything—variables, logic, loops, functions, objects, and data analysis—into a single **Mini-Project Integration**.

11 Glossary of Terms

- **Aggregation:** The process of summarizing data into a single value (e.g., sum, mean, count).
- **CSV:** Comma Separated Values; a standard file format for exchanging tabular data.
- **DataFrame:** The primary data structure in Pandas, representing a table with rows and columns.
- **Grouping:** Organizing data into categories before performing an analysis.
- **Matplotlib:** The standard Python library for creating 2D plots and graphs.
- **NaN:** Not a Number; used by Pandas to represent missing or null data.
- **Outlier:** A data point that differs significantly from other observations, potentially indicating a sensor error.
- **Pandas:** An open-source data analysis and manipulation tool for Python.
- **Vectorized Operation:** A mathematical operation that is applied to an entire array or column simultaneously, without explicit loops.
- **Visualization:** The graphical representation of information and data.

12 Extended Self-Assessment Exercises

12.1 Exercise 1: Irrigation Data Analysis

A CSV file named `irrigation_log.csv` has three columns: `Date`, `Block_ID`, and `Water_Applied_L`. **Task:** 1. Load the file into a Pandas DataFrame. 2. Calculate the total water applied across the whole farm. 3. Calculate the average water applied per `Block_ID` using `groupby`.

12.2 Exercise 2: Weather Data Cleaning

You have a DataFrame `df_weather` with a `Relative_Humidity` column. Some readings are `-999` (an error code used by the weather station). **Task:** 1. Use filtering to replace all `-999` values with `NaN`. 2. Use `fillna()` to replace the `NaN`s with the average humidity of the dataset.

12.3 Exercise 3: The Growth Curve Plot

Task: Create a line plot representing the growth of a maize plant.

- X-axis: Weeks [1, 2, 3, 4, 5, 6, 7, 8]
- Y-axis: Height [10, 25, 45, 80, 120, 160, 200, 210] cm
- Requirements: Red dashed line, blue square markers, grid enabled, and a title "Maize Height vs. Time".

12.4 Exercise 4: Machinery Efficiency Heatmap (Advanced)

Explain in five sentences why an engineer would prefer to use a scatter plot with color-coding (based on load) rather than a simple line plot when analyzing tractor fuel data.

End of Lecture 9 Compendium